DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT REPORT

on

HANDWRITTEN DIGIT RECOGNITION USING A DEEP NEURAL NETWORK

Submitted by

Name	: Dhansheera Sadak
Register Number	: 711524BCS031
Year / Branch	: III Year – B.E. CSE
Section	: B
Subject	: Deep Neural Networks
Academic Year	: 2026 – 2027

1. Abstract

Handwritten digit recognition is a classic problem in computer vision and pattern recognition in which a machine must identify a digit from 0 to 9 written by a human. This mini-project builds a deep neural network (DNN) that performs this task using the MNIST dataset of 70,000 grayscale images, each of size 28 × 28 pixels.

The images are normalized and flattened into 784-element vectors, which are passed through a fully connected network with two hidden layers (128 and 64 neurons) using the ReLU activation, a dropout layer for regularization and a 10-neuron Softmax output layer. The network is trained with the Adam optimizer, which applies gradient descent on the categorical cross-entropy loss, with the gradients obtained through backpropagation.

After 15 epochs with a batch size of 128, the model reached a test accuracy of 97.66% on 10,000 unseen images, with a macro-averaged F1-score of 0.9765. The report presents the theory, architecture, TensorFlow/Keras implementation, training curves, confusion matrix and a discussion of results, limitations and future work.

Keywords: *Deep Learning, Neural Network, MNIST, Handwritten Digit Recognition, ReLU, Softmax, Backpropagation, Keras.*

2. Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with several layers to learn useful representations directly from raw data. Instead of hand-crafting features, a deep network discovers them during training: early layers capture simple patterns such as edges and strokes, while later layers combine them into more abstract shapes. This ability has made deep learning the leading approach in image recognition, speech processing, language translation and many other fields.

An artificial neural network is loosely inspired by the human brain. It is built from small computing units called neurons that are organized into layers and connected by weighted links. Each neuron multiplies its inputs by weights, adds a bias and passes the result through a non-linear activation function. Training adjusts the weights so that the network's outputs match the known answers for a large set of examples.

Digit recognition is an ideal first project in deep learning because the data are small, clean and well labelled, results can be compared with a large body of published work, and the model trains in a couple of minutes on an ordinary computer. The knowledge gained here – data preprocessing, layer design, loss functions, optimization and evaluation – transfers directly to larger image-classification problems.

3. Problem Statement and Objectives

3.1 Problem statement

To design, train and evaluate a deep neural network that takes an image of a single handwritten digit as input and correctly predicts the digit (0–9) with high accuracy, without any manual feature engineering.

3.2 Why automated digit recognition is useful

Large amounts of handwritten numeric information are still produced every day – postal codes, bank cheques, forms, exam marks and meter readings. Typing this data manually is slow, expensive and error-prone. Automatic recognition speeds up processing, reduces human error and allows paper records to be digitized at scale.

3.3 Objectives

- To understand the working of artificial neurons, layers, activation functions and forward propagation.
- To preprocess the MNIST images by normalization and reshaping.
- To build a multi-layer (deep) neural network using TensorFlow/Keras.
- To train the network using backpropagation and gradient-descent-based optimization with suitable epochs and batch size.
- To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- To analyse the errors made by the model and suggest improvements.

4. Literature Review

Many approaches have been used for handwritten digit recognition since the creation of the MNIST dataset by LeCun and colleagues. They can be grouped as follows:

Approach	Idea	Typical MNIST accuracy
k-Nearest Neighbours	Classifies an image by the labels of the closest training images using pixel distance.	≈ 96–97%
Support Vector Machine	Finds a maximum-margin boundary, often with a kernel such as RBF.	≈ 98–99%
Multi-layer perceptron (this work)	Fully connected layers learn features from flattened pixels.	≈ 97–98.5%
Convolutional Neural Network (LeNet-5 and successors)	Convolution and pooling exploit the 2-D structure of images.	≈ 99%+
Ensembles and augmentation-based CNNs	Combine several networks and use distorted training images.	≈ 99.7%

Early work by LeCun et al. (1998) showed that gradient-based learning with convolutional networks outperforms classical methods on digit recognition. Later ideas such as ReLU, dropout and the Adam optimizer made deeper networks easier to train.

5. Deep Neural Network Fundamentals

5.1 Neurons and layers

A neuron computes a weighted sum of its inputs and adds a bias: $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$. The result is passed through an activation function f to give the output $a = f(z)$. Neurons are arranged in layers:

- **Input layer –** receives the 784 pixel values of an image; it performs no computation.
- **Hidden layers –** learn intermediate features and patterns of digits such as loops, curves and straight strokes.
- **Output layer –** contains one neuron per class (10) and produces the final prediction.

5.2 Activation functions

Without a non-linear activation, a stack of layers would collapse into a single linear transformation. This project uses two activations:

- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$. It is fast to compute, avoids the vanishing-gradient problem for positive inputs and is used in the hidden layers.
- **Softmax:** $\sigma(z_i) = e^{z_i} / \Sigma_j e^{z_j}$. It converts the ten output scores into probabilities that sum to 1; the digit with the highest probability is the prediction.

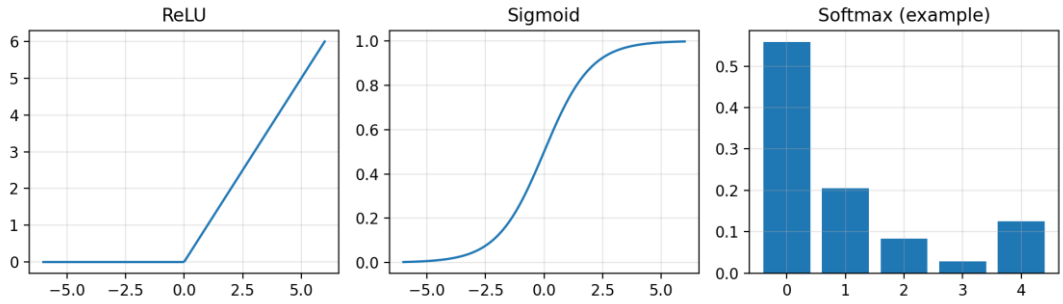


Figure 1: ReLU, Sigmoid (for comparison) and an example Softmax output

5.3 Forward propagation

In forward propagation the input vector is passed layer by layer: $a^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)})$, $a^{(2)} = \text{ReLU}(W^{(2)}a^{(1)} + b^{(2)})$ and $\hat{y} = \text{Softmax}(W^{(3)}a^{(2)} + b^{(3)})$. The vector $\hat{y}$ holds the predicted probability of each digit.

(a) Define value education and explain why it is needed.

(b) Distinguish between happiness and prosperity. Which one is a person missing if they have wealth and success but still feel restless and empty?

5.4 Gradient descent and optimizer

Gradient descent updates each weight in the direction that reduces the loss: $w \leftarrow w - \eta \cdot \partial L/\partial w$, where η is the learning rate. In mini-batch gradient descent the gradient is estimated on a small batch of images, which is faster and less noisy than using one image at a time. This project uses the Adam optimizer, an extension of gradient descent that keeps running averages of the gradients and their squares to adapt the learning rate of each parameter (default $\eta = 0.001$).

6. Proposed System and Architecture

The proposed system follows the standard supervised-learning pipeline: the dataset is loaded and preprocessed, a deep neural network is defined, it is trained using backpropagation and finally evaluated on unseen test images.

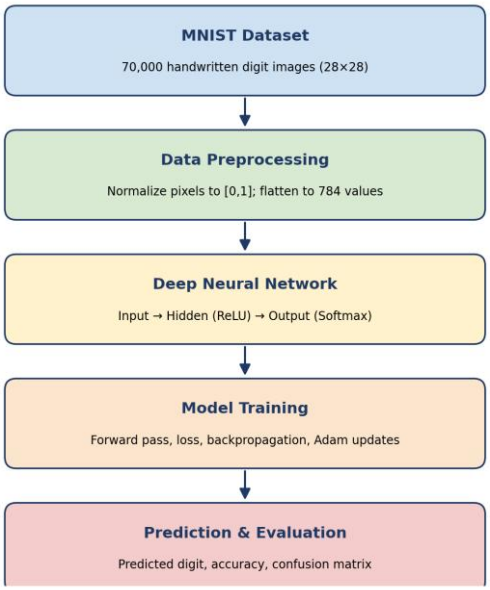


Figure 2: Project workflow

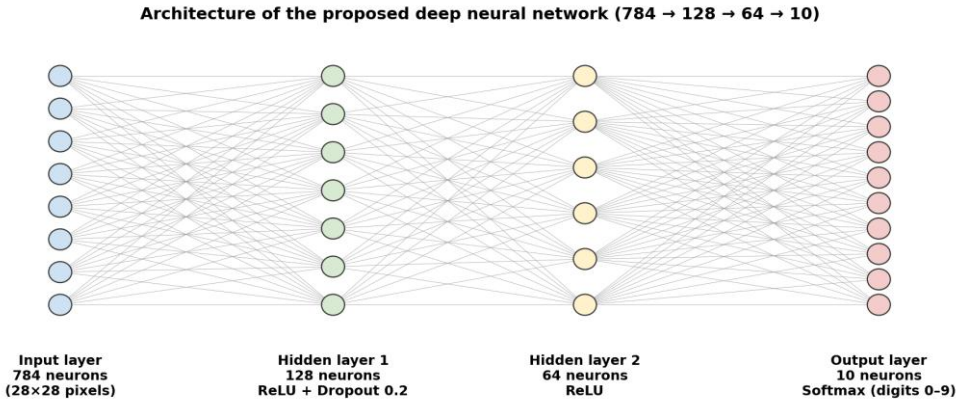


Figure 3: Network architecture (a representative subset of neurons is drawn per layer)

6.1 Layer-wise model summary

Layer	Output shape	Parameters	Activation
Flatten	(784)	0	–
Dense 1	(128)	100,480	ReLU
Dropout (0.2)	(128)	0	–
Dense 2	(64)	8,256	ReLU
Dense 3 (output)	(10)	650	Softmax
Total trainable		109,386	

7. Dataset and Preprocessing

7.1 MNIST dataset

The Modified National Institute of Standards and Technology (MNIST) dataset contains 70,000 grayscale images of handwritten digits collected from American Census Bureau employees and high-school students. Each image is 28 × 28 pixels, and every pixel holds an intensity from 0 (black background) to 255 (white stroke).

Subset	Images	Use
Training	60,000 (54,000 train + 6,000 validation)	Learning and tuning
Test	10,000	Final unseen evaluation
Classes	10 (digits 0–9)	Output labels

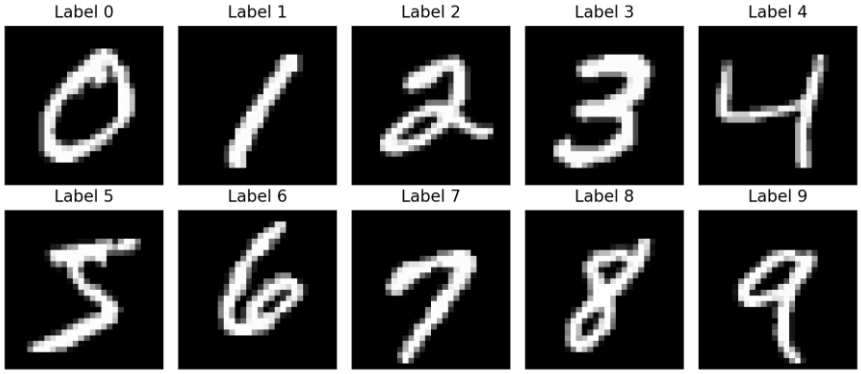


Figure 4: One sample image from each class of the MNIST dataset

7.2 Preprocessing

- **Normalization:** pixel values are divided by 255 so that they lie between 0 and 1, which speeds up and stabilizes gradient descent.
- **Reshaping:** each 28 × 28 image is flattened into a vector of 784 values for the fully connected network (done by the Flatten layer).
- **Labels:** kept as integers 0–9 and used with the sparse categorical cross-entropy loss.

8. Implementation

The model is implemented in Python using TensorFlow/Keras. The complete program is given below. The call keras.datasets.mnist.load_data() loads the dataset on Colab or a local machine; the results in this report come from running the same network, settings and seed on a local MNIST copy.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import confusion_matrix, classification_report


tf.keras.utils.set random seed(42)


# 1. Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()


# 2. Preprocessing: normalize pixel values to the range [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0


# 3. Build the deep neural network
model = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),                          # 28x28 -> 784
    layers.Dense(128, activation='relu'),      # hidden layer 1
    layers.Dropout(0.2),                       # regularization
    layers.Dense(64, activation='relu'),       # hidden layer 2
    layers.Dense(10, activation='softmax')     # output layer
])
model.summary()


# 4. Compile: optimizer, loss function and metric
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])


# 5. Train (forward pass + backpropagation + Adam updates)
history = model.fit(x train, y train, epochs=15,
                    batch size=128, validation split=0.1)


# 6. Evaluate on the unseen test set
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc)


# 7. Predictions, confusion matrix and classification report
y_pred = np.argmax(model.predict(x_test), axis=1)
print(confusion matrix(y test, y pred))
print(classification report(y test, y pred, digits=4))


# 8. Predict a single handwritten digit image (28x28, white digit on black)
def predict_digit(img_array):
    img = img_array.astype('float32').reshape(1, 28, 28) / 255.0
    probs = model.predict(img)[0]
    return int(np.argmax(probs)), float(np.max(probs))


digit, conf = predict_digit(x_test[0] * 255)
print(f'Predicted digit: {digit} (confidence {conf:.2%})')
```

9. Results and Discussion

9.1 Training curves

Training accuracy rose from 88.4% in the first epoch to 98.76% in the fifteenth, while the validation accuracy settled at around 98%. The training loss decreased steadily from 0.406 to 0.037. The validation loss flattens after about epoch 8, and the small gap between training and validation curves shows mild over-fitting, which dropout keeps under control.

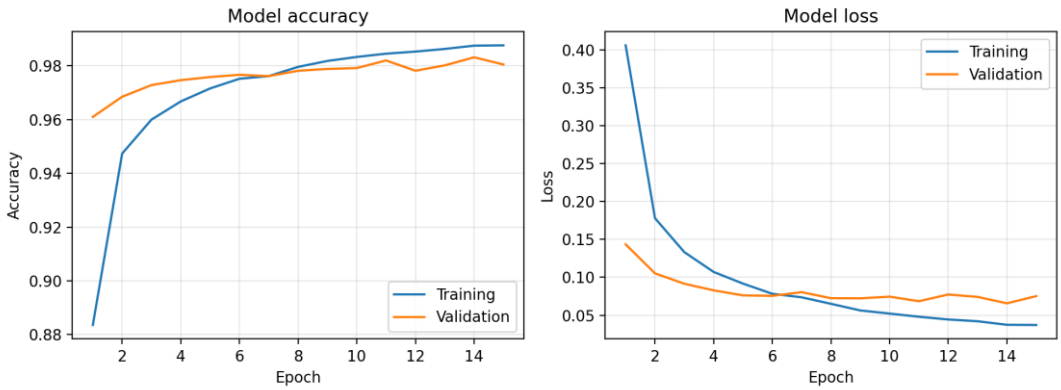


Figure 5: Training and validation accuracy and loss over 15 epochs

9.2 Test performance

Measure	Value	Note
Test accuracy	97.66%	10,000 unseen images
Test loss	0.0763	Cross-entropy
Macro precision	0.9766	Average over 10 classes
Macro recall	0.9765	Average over 10 classes
Macro F1-score	0.9765	Average over 10 classes
Misclassified images	234 of 10,000	2.34% error rate

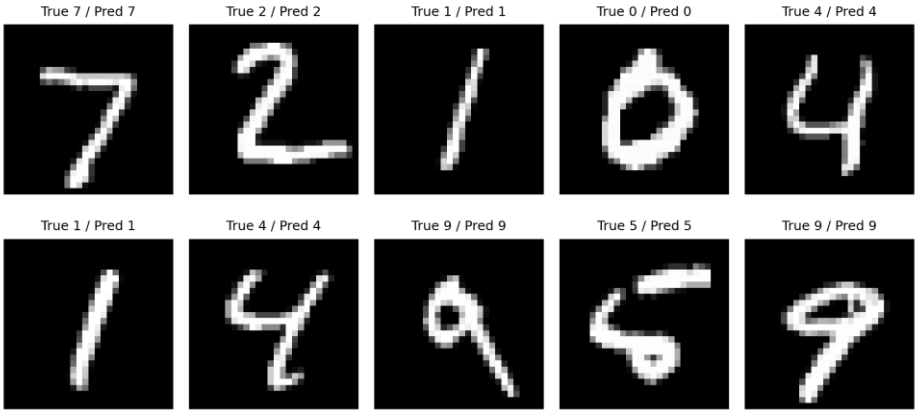


Figure 6: Sample predictions on the first ten test images

9.3 Confusion matrix

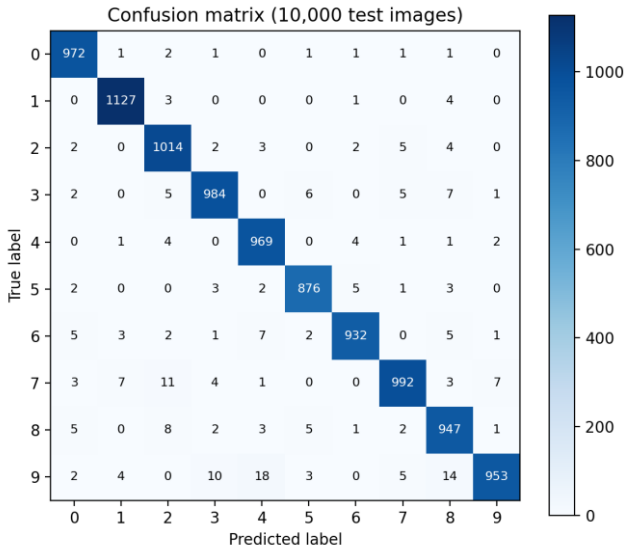


Figure 7: Confusion matrix on the test set (rows: true digit, columns: predicted digit)

The strong diagonal shows that most digits are recognized correctly. The most frequent mistakes are 9 predicted as 4 (18 images), 9 as 8 (14), 7 as 2 (11) and 9 as 3 (10). These digit pairs have similar shapes when written quickly or with a closed loop, which explains the confusion.

9.4 Class-wise precision, recall and F1-score

Digit	Precision	Recall	F1-score	Support
0	0.9789	0.9918	0.9853	980
1	0.9860	0.9930	0.9895	1135
2	0.9666	0.9826	0.9745	1032
3	0.9772	0.9743	0.9757	1010
4	0.9661	0.9868	0.9763	982
5	0.9810	0.9821	0.9815	892
6	0.9852	0.9729	0.9790	958
7	0.9802	0.9650	0.9725	1028
8	0.9575	0.9723	0.9648	974
9	0.9876	0.9445	0.9656	1009

Digit 1 is recognized best (F1 = 0.9895) because of its simple shape, while digits 8 and 9 are the hardest (F1 ≈ 0.965), as they are often confused with other loop-shaped digits. The discussion confirms that a modest fully connected network can already learn the main structure of handwritten digits.

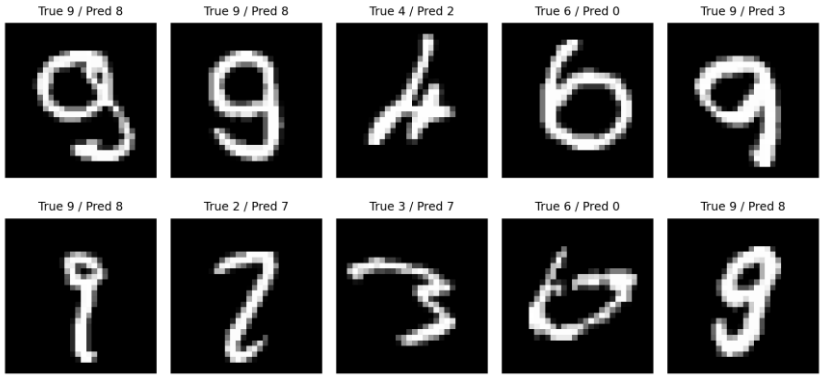


Figure 8: Examples of misclassified test images (true label / predicted label)

10. Advantages, Limitations and Applications

10.1 Advantages

- Simple architecture that is easy to understand, implement and train in a few minutes.
- No manual feature extraction – the network learns features from raw pixels.
- High accuracy (≈ 97.7%) with only about 109 thousand parameters.
- Dropout and Adam optimizer give stable training and reduce over-fitting.
- Built with open-source tools (Python, TensorFlow/Keras).

10.2 Limitations

- Flattening the image discards the 2-D spatial structure, so the model is less accurate than a CNN.
- It is sensitive to the position, size and slant of the digit.
- It is trained on clean, centred MNIST digits and may not generalize to noisy real-world photos without extra preprocessing.
- It recognizes only single digits, not multi-digit numbers or letters.

10.3 Applications

- Postal-code reading and automatic mail sorting.
- Bank cheque amount and account-number processing.
- Digitizing handwritten forms, exam sheets and survey data.
- Reading meter values and number plates; assistive tools for education.

11. Conclusion and Future Enhancement

A deep neural network with two ReLU hidden layers, dropout and a Softmax output was built and trained on the MNIST dataset using TensorFlow/Keras. The network learned through forward propagation, cross-entropy loss, backpropagation and Adam-based gradient descent and achieved a test accuracy of 97.66% with a macro F1-score of 0.9765. The project demonstrates how the core concepts of deep neural networks fit together in a practical application.

Future enhancements include: using a Convolutional Neural Network to exceed 99% accuracy; data augmentation (rotation, shift, zoom) for robustness; hyper-parameter tuning and early stopping; recognizing multi-digit numbers and handwritten letters (EMNIST); and deploying the model as a web or mobile application that accepts a drawn or uploaded digit image.

12. References

[1] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] Y. LeCun, C. Cortes and C. J. C. Burges, "The MNIST database of handwritten digits." http://yann.lecun.com/exdb/mnist/

[3] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. MIT Press, 2016.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[5] N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[6] TensorFlow/Keras documentation. https://www.tensorflow.org/api_docs